

Shell script

THEORY:

A shell program (shell script) is a text file that contains standard UNIX and shell commands. Each line in a shell program contains a single UNIX command exactly as if you had typed them in yourself. The difference is that you can execute all the commands in a shell program simply by running the shell program rather than typing all the commands within.

The shell script has following commands,

1. File creation
\$ cat>file.sh
2. File execution
1st Method: (i) \$ chmod +x file.sh
(ii) \$ file.sh
2nd Method: \$ sh file.sh

READ STATEMENT:

To get input from the keyboard, the read command is used. The read command takes input from the keyboard and assigns it to a variable.

EXAMPLE:

```
echo "Enter some text > "  
read text  
echo "You entered: $text"
```

OUTPUT:

```
Enter some text > operating system  
You entered: operating system
```

POSITIONAL PARAMETERS:

The command-line arguments \$1, \$2, \$3,...\$9 are positional parameters, with \$0 pointing to the actual command, program, shell script, or function and \$1, \$2, \$3, ...\$9 as the arguments to the command.

Variable	Meaning
\$0	the name of the shell program
\$1 to \$9	the first to ninth parameter
\$#	the number of parameters
\$*	all the parameters passed represented as a single word with individual parameters separated
\$@	all the parameters passed with each parameter as a separate word

\$?	hold the exit status of the previous command
------------	---

EXAMPLE:

```
echo "First Parameter : $1"
echo "Second Parameter : $2"
echo "Quoted Values: $@"
echo "Quoted Values: $*"
echo "Total Number of Parameters : $#"
```

OUTPUT:

```
$sh file1.sh abc xyz
First Parameter : abc
Second Parameter : xyz
Quoted Values: abc xyz
Quoted Values: abc xyz
Total Number of Parameters : 2
```

EXIT STATUS

The \$? Variable represents the exit status of the previous command. Exit status is a numerical value returned by every command upon its completion. As a rule, most commands return an exit status of 0 if they were successful, and 1 if they were unsuccessful. Some commands return additional exit statuses for particular reasons. For example, some commands differentiate between kinds of errors and will return various exit values depending on the specific type of failure.

OPERATORS:

- Arithmetic Operators.
- Relational Operators.
- Boolean Operators.
- String Operators.
- File Test Operators.

ARITHMETIC OPERATORS

Assume variable a holds 10 and variable b holds 20 then,

Operator	Description	Example
+	Addition - Adds values on either side of the operator	`expr \$a + \$b` will give 30
-	Subtraction - Subtracts right hand operand from left hand operand	`expr \$a - \$b` will give -10

*	Multiplication - Multiplies values on either side of the operator	`expr \$a \* \$b` will give 200
/	Division - Divides left hand operand by right hand operand	`expr \$b / \$a` will give 2
%	Modulus - Divides left hand operand by right hand operand and returns remainder	`expr \$b % \$a` will give 0
=	Assignment - Assign right operand in left operand	a=\$b would assign value of b into a
==	Equality - Compares two numbers, if both are same then returns true.	[\$a == \$b] would return false.
!=	Not Equality - Compares two numbers, if both are different then returns true.	[\$a != \$b] would return true.

EXAMPLE:

a=10

b=20

**val=`expr \$a + \$b`
echo "a + b : \$val"**

**val=`expr \$a - \$b`
echo "a - b : \$val"**

**val=`expr \$a \\* \$b`
echo "a * b : \$val"**

**val=`expr \$b / \$a`
echo "b / a : \$val"**

**val=`expr \$b % \$a`
echo "b % a : \$val"**

OUTPUT:

a + b : 30

a - b : -10

a * b : 200

b / a : 2

b % a : 0

RELATIONAL OPERATORS:

Assume variable a holds 10 and variable b holds 20 then,

Operator	Description	Example
-eq	Checks if the value of two operands are equal or not, if yes then condition becomes true.	[\$a -eq \$b] is not true.
-ne	Checks if the value of two operands are equal or not, if values are not equal then condition becomes true.	[\$a -ne \$b] is true.
-gt	Checks if the value of left operand is greater than the value of right operand, if yes then condition becomes true.	[\$a -gt \$b] is not true.
-lt	Checks if the value of left operand is less than the value of right operand, if yes then condition becomes true.	[\$a -lt \$b] is true.
-ge	Checks if the value of left operand is greater than or equal to the value of right operand, if yes then condition becomes true.	[\$a -ge \$b] is not true.
-le	Checks if the value of left operand is less than or equal to the value of right operand, if yes then condition becomes true.	[\$a -le \$b] is true.

EXAMPLE:

a=10

b=20

if [\$a -eq \$b]

then

echo "\$a -eq \$b : a is equal to b"

else

echo "\$a -eq \$b: a is not equal to b"

fi

if [\$a -ne \$b]

then

```
    echo "$a -ne $b: a is not equal to b"
else
    echo "$a -ne $b : a is equal to b"
fi
```

```
if [ $a -gt $b ]
then
    echo "$a -gt $b: a is greater than b"
else
    echo "$a -gt $b: a is not greater than b"
fi
```

```
if [ $a -lt $b ]
then
    echo "$a -lt $b: a is less than b"
else
    echo "$a -lt $b: a is not less than b"
fi
```

```
if [ $a -ge $b ]
then
    echo "$a -ge $b: a is greater or equal to b"
else
    echo "$a -ge $b: a is not greater or equal to b"
fi
```

```
if [ $a -le $b ]
then
    echo "$a -le $b: a is less or equal to b"
else
    echo "$a -le $b: a is not less or equal to b"
fi
```

OUTPUT:

```
10 -eq 20: a is not equal to b
10 -ne 20: a is not equal to b
10 -gt 20: a is not greater than b
10 -lt 20: a is less than b
10 -ge 20: a is not greater or equal to b
10 -le 20: a is less or equal to b
```

BOOLEAN OPERATORS:

Assume variable a holds 10 and variable b holds 20 then,

Operator	Description	Example
!	This is logical negation. This inverts a true condition into false and vice versa.	[! false] is true.
-o	This is logical OR. If one of the operands is true then condition would be true.	[\$a -lt 20 -o \$b -gt 100] is true.
-a	This is logical AND. If both the operands are true then condition would be true otherwise it would be false.	[\$a -lt 20 -a \$b -gt 100] is false.

EXAMPLE:

a=10

b=20

if [\$a != \$b]

then

echo "\$a != \$b : a is not equal to b"

else

echo "\$a != \$b: a is equal to b"

fi

if [\$a -lt 100 -a \$b -gt 15]

then

echo "\$a -lt 100 -a \$b -gt 15 : returns true"

else

echo "\$a -lt 100 -a \$b -gt 15 : returns false"

fi

if [\$a -lt 100 -o \$b -gt 100]

then

echo "\$a -lt 100 -o \$b -gt 100 : returns true"

else

echo "\$a -lt 100 -o \$b -gt 100 : returns false"

fi

if [\$a -lt 5 -o \$b -gt 100]

then

```

    echo "$a -lt 100 -o $b -gt 100 : returns true"
else
    echo "$a -lt 100 -o $b -gt 100 : returns false"
fi

```

OUTPUT:

```

10 != 20 : a is not equal to b
10 -lt 100 -a 20 -gt 15 : returns true
10 -lt 100 -o 20 -gt 100 : returns true
10 -lt 5 -o 20 -gt 100 : returns false

```

STRING OPERATORS:

Assume variable a holds "abc" and variable b holds "efg" then,

Operator	Description	Example
=	Checks if the value of two operands are equal or not, if yes then condition becomes true.	[\$a = \$b] is not true.
!=	Checks if the value of two operands are equal or not, if values are not equal then condition becomes true.	[\$a != \$b] is true.
-z	Checks if the given string operand size is zero. If it is zero length then it returns true.	[-z \$a] is not true.
-n	Checks if the given string operand size is non-zero. If it is non-zero length then it returns true.	[-n \$a] is not false.
str	Check if str is not the empty string. If it is empty then it returns false.	[\$a] is not false.

EXAMPLE:

```

a="abc"
b="efg"

```

```

if [ $a = $b ]
then
    echo "$a = $b : a is equal to b"
else
    echo "$a = $b: a is not equal to b"
fi

```

```

if [ $a != $b ]
then
    echo "$a != $b : a is not equal to b"
else
    echo "$a != $b: a is equal to b"
fi

```

```

if [ -z $a ]
then
    echo "-z $a : string length is zero"
else
    echo "-z $a : string length is not zero"
fi

```

```

if [ -n $a ]
then
    echo "-n $a : string length is not zero"
else
    echo "-n $a : string length is zero"
fi

```

```

if [ $a ]
then
    echo "$a : string is not empty"
else
    echo "$a : string is empty"
fi

```

OUTPUT:

```

abc = efg: a is not equal to b
abc != efg : a is not equal to b
-z abc : string length is not zero
-n abc : string length is not zero
abc : string is not empty

```

FILE TEST OPERATORS:

There are following operators to test various properties associated with a file. Assume a variable file holds an existing file name "test" whose size is 100 bytes and has read, write and execute permission on,

Operator	Description	Example
----------	-------------	---------

-d file	Check if file is a directory if yes then condition becomes true.	[-d \$file] is not true.
-f file	Check if file is an ordinary file as opposed to a directory or special file if yes then condition becomes true.	[-f \$file] is true.
-r file	Checks if file is readable if yes then condition becomes true.	[-r \$file] is true.
-w file	Check if file is writable if yes then condition becomes true.	[-w \$file] is true.
-x file	Check if file is execute if yes then condition becomes true.	[-x \$file] is true.
-s file	Check if file has size greater than 0 if yes then condition becomes true.	[-s \$file] is true.
-e file	Check if file exists. Is true even if file is a directory but exists.	[-e \$file] is true.

EXAMPLE:

file="/home/bca/a.sh" (you can assign path of any file)

```

if [ -r $file ]
then
    echo "File has read access"
else
    echo "File does not have read access"
fi

if [ -w $file ]
then
    echo "File has write permission"
else
    echo "File does not have write permission"
fi

if [ -x $file ]
then
    echo "File has execute permission"
else

```

```
echo "File does not have execute permission"
```

```
fi
```

```
if [ -f $file ]
```

```
then
```

```
echo "File is an ordinary file"
```

```
else
```

```
echo "This is sepcial file"
```

```
fi
```

```
if [ -d $file ]
```

```
then
```

```
echo "File is a directory"
```

```
else
```

```
echo "This is not a directory"
```

```
fi
```

```
if [ -s $file ]
```

```
then
```

```
echo "File size is not zero"
```

```
else
```

```
echo "File size is zero"
```

```
fi
```

```
if [ -e $file ]
```

```
then
```

```
echo "File exists"
```

```
else
```

```
echo "File does not exist"
```

```
fi
```

OUTPUT:

File has read access

File has write permission

File has execute permission

File is an ordinary file

This is not a directory

File size is zero

File exists

EXPR:

The shell does not have any computing features at all, it has to rely on the **EXPR** command for that purpose. It can perform arithmetic operations on integers and also manipulates strings to a limited extent.

```
$ z=`expr $z + 1`
```

```
$ echo $z
```

```
6
```